- `scanLeft` and `scanRight`—Like `foldLeft` and `foldRight`, but they return the `List` of partial results rather than just the final accumulated value

We recommend that you look through the Scala API documentation after finishing this chapter, to see what other functions there are. If you find yourself writing an explicit recursive function for doing some sort of list manipulation, check the `List` API to see if something like the function you need already exists.

3.4.2 Loss of efficiency when assembling list functions from simpler components

One of the problems with `List` is that, although we can often express operations and algorithms in terms of very general-purpose functions, the resulting implementation isn't always efficient—we may end up making multiple passes over the same input, or else have to write explicit recursive loops to allow early termination.

EXERCISE 3.24

Hard: As an example, implement `hasSubsequence` for checking whether a `List` contains another `List` as a subsequence. For instance, `List(1,2,3,4)` would have `List(1,2)`, `List(2,3)`, and `List(4)` as subsequences, among others. You may have some difficulty finding a concise purely functional implementation that is also efficient. That's okay. Implement the function however comes most naturally. We'll return to this implementation in chapter 5 and hopefully improve on it. Note: Any two values `x` and `y` can be compared for equality in Scala using the expression `x == y`.

```
def hasSubsequence[A](sup: List[A], sub: List[A]): Boolean
```

3.5 Trees

`List` is just one example of what's called an *algebraic data type (ADT)*. (Somewhat confusingly, ADT is sometimes used elsewhere to stand for *abstract data type*.) An ADT is just a data type defined by one or more data constructors, each of which may contain zero or more arguments. We say that the data type is the *sum* or *union* of its data constructors, and each data constructor is the *product* of its arguments, hence the name *algebraic* data type.[14]

[14] The naming is not coincidental. There's a deep connection, beyond the scope of this book, between the "addition" and "multiplication" of types to form an ADT and addition and multiplication of numbers.

Tuple types in Scala

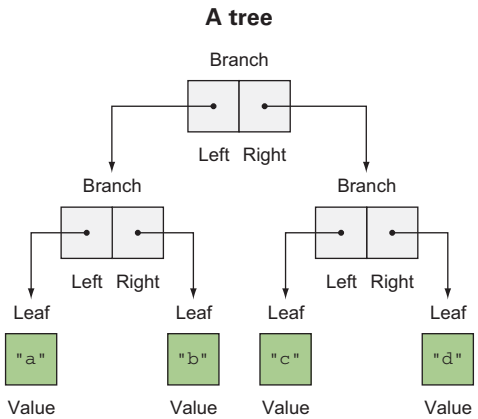
Pairs and tuples of other arities are also algebraic data types. They work just like the ADTs we've been writing here, but have special syntax:

```
scala> val p = ("Bob", 42)
p: (java.lang.String, Int) = (Bob,42)

scala> p._1
res0: java.lang.String = Bob

scala> p._2
res1: Int = 42

scala> p match { case (a,b) => b }
res2: Int = 42
```

In this example, `("Bob", 42)` is a pair whose type is `(String,Int)`, which is syntactic sugar for `Tuple2[String,Int]` (API link: http://mng.bz/1F2N). We can extract the first or second element of this pair (a `Tuple3` will have a method `_3`, and so on), and we can pattern match on this pair much like any other `case class`. Higher arity tuples work similarly—try experimenting with them in the REPL if you're interested.

Algebraic data types can be used to define other data structures. Let's define a simple binary tree data structure:

```
sealed trait Tree[+A]
case class Leaf[A](value: A) extends Tree[A]
case class Branch[A](left: Tree[A], right: Tree[A]) extends Tree[A]
```



Pattern matching again provides a convenient way of operating over elements of our ADT. Let's try writing a few functions.

EXERCISE 3.25

Write a function `size` that counts the number of nodes (leaves and branches) in a tree.

EXERCISE 3.26

Write a function `maximum` that returns the maximum element in a `Tree[Int]`. (Note: In Scala, you can use `x.max(y)` or `x max y` to compute the maximum of two integers `x` and `y`.)

EXERCISE 3.27

Write a function `depth` that returns the maximum path length from the root of a tree to any leaf.

EXERCISE 3.28

Write a function `map`, analogous to the method of the same name on `List`, that modifies each element in a tree with a given function.

ADTs and encapsulation

One might object that algebraic data types violate encapsulation by making public the internal representation of a type. In FP, we approach concerns about encapsulation differently—we don't typically have delicate mutable state which could lead to bugs or violation of invariants if exposed publicly. Exposing the data constructors of a type is often fine, and the decision to do so is approached much like any other decision about what the public API of a data type should be.[15]

We do typically use ADTs for situations where the set of cases is *closed* (known to be fixed). For `List` and `Tree`, changing the set of data constructors would significantly change what these data types are. `List` *is* a singly linked list—that is its nature—and the two cases `Nil` and `Cons` form part of its useful public API. We can certainly write code that deals with a more abstract API than `List` (we'll see examples of this later in the book), but this sort of information hiding can be handled as a separate layer rather than being baked into `List` directly.

[15] It's also possible in Scala to expose patterns like `Nil` and `Cons` independent of the actual data constructors of the type.